AP React – Partie 1
Authentification

Introduction

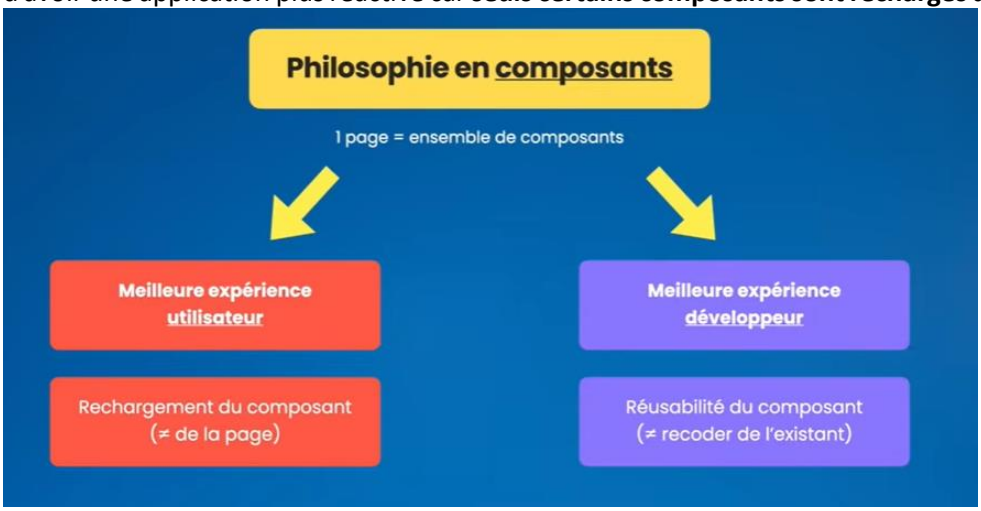
Dans les séances à venir, vous allez relever un défi concret : moderniser l'application de gestion des notes de frais pour l'organisation GSB. Votre mission ? Remplacer l'application développée en Laravel avec un front-end basé sur Bootstrap, en une solution plus interactive et performante grâce à React.

L'objectif est clair : améliorer l'expérience utilisateur en exploitant les atouts de React pour créer une interface plus fluide, réactive et intuitive. Vous mettrez ainsi en pratique vos compétences en intégration front-end et en développement d'applications full-stack, tout en répondant aux besoins réels d'une entreprise.

L'objectif n'est pas de réaliser l'application le plus vite possible mais de bien comprendre la logique du code avec React. A la fin de cette partie, vous devez être à l'aise avec les notions suivantes et la façon de les implémenter : les composants, les props, les event listeners.

Quelques rappels sur React

React est un framework Javascript basé sur une philosophie en composants qui permet de maintenir plus facilement le code (pour les développeurs) car il n'y a pas de long fichier de code : **le code est divisé en plusieurs composants**, stockés chacun dans un fichier différent. Cette philosophie permet aussi d'avoir une application plus réactive car **seuls certains composants sont rechargés** à chaque interaction.



Un composant est un bloc fonctionnel (défini comme une fonction). Il prend en argument des props, qui sont des données qu'il reçoit d'un autre composant.
Un composant est toujours composé de **3 blocs, codés dans cet ordre** :

- **le state** : l'état, c'est-à-dire les données qu'il gère et manipule.
- **les comportements** : les actions qui seront exécutées en réponse aux interactions de l'utilisateur et qui modifieront l'état.
- **l'affichage** (ou rendering) : du code JSX (une sorte de HTML avec du Javascript) décrivant ce qui sera affiché dans la page lorsque le composant sera rendu (rendered).

Dans votre code, un composant ressemblera à cela :

```
function MonComposant(props){
  // state (état, données)

  // comportements

  // affichage (render)
  return (/*code JSX*/)
}
```

Commençons par créer le projet gsb-frais-react sur le boot que vous utilisez habituellement en cours de React. Vous aurez besoin de VSCode avec yarn.

1. Lancer VSCode et ouvrir le dossier dans lequel vous voulez stocker votre code

Nous allons utiliser Create React App (CRA), la boîte à outils créée par Facebook, qui est la référence pour initier un projet React.

2. Ouvrir un Terminal (onglet Terminal > Nouveau) et exécuter la commande suivante pour créer un projet React (si vous utilisez npm à la place de yarn, simplement remplacer yarn par npm dans les commandes) :

   ```
   yarn create react-app gsb-frais-react
   ```

3. Vérifier le bon fonctionnement en lançant l'application et en vérifiant qu'elle se lance dans le navigateur.

   ```
   cd gsb-frais-react
   yarn start
   ```

   Pour rappel, un projet initialisé avec CRA possède toujours :
   - un fichier index.html qui est le template où vivra notre app React ;
   - un package.json qui liste les dépendances et les scripts ;
   - un fichier index.js dans lequel notre app React est initialisée, et greffée au HTML.

   Plus tard, nous utiliserons d'autres commandes :
   - `yarn run build` vous permettra de créer un build avec votre code transformé et minifié, si vous devez déployer votre application en production (la mettre en ligne, par exemple) ;
   - `yarn test` pour exécuter les tests.

Vous allez développer une Single Page Application, avec un système de routage pour passer d'un composant à l'autre.

Dans cette partie, nous allons ajouter les fonctionnalités suivantes :

- **Authentification** : Gérée via un contexte (AuthContext).
- **Barre de menu** : Affiche des liens différents selon que l'utilisateur est connecté ou non.
- **Connexion/Déconnexion** : Logique basique, que nous remplacerons par une requête à une API dans les séances suivantes.
- **Route protégée** : La page Dashboard n'est accessible qu'aux utilisateurs connectés.

Étape 1 : Préparation de l'environnement

1. Installer react-router-dom :

   ```
   yarn add react-router-dom
   ```

2. Créer les dossiers suivants dans /src :

```
/src
  /components
  /pages
  /context
  /styles
```

Étape 2 : Mise en place du routage

1. **Créer les pages** :
   - Dans /src/pages, créez trois fichiers :
     - Home.jsx (page d'accueil)
     - Login.jsx (page de connexion)
     - Dashboard.jsx (page protégée)
2. **Configurer le routage** :
   - Dans App.js, importez BrowserRouter, Routes, Route et Link depuis react-router-dom.
   - Déclarez les routes pour /, /login et /dashboard. Par exemple, pour la page de connexion, ajoutez l'import de la page dans App.js : import Login from './pages/Login';
3. **Créer les composants :**
   - Créer les composants Home, Login et Dashboard, avec un code minimal. Par exemple, pour Home, on peut simplement afficher « Bienvenue » :

```
function Home(){
    return (
        <h1>Bienvenue</h1>
    );
}

export default Home;
```

On affichera par exemple « Page de connexion » dans Login et « Tableau de bord » dans Dashboard.

4. **Tester la navigation** :
   - Ajoutez des liens temporaires dans App.js pour vérifier que les pages s'affichent correctement :

```
function App() {
  return (
    <BrowserRouter>
      {/* Liens temporaires pour tester les routes */}
      <div style={{ padding: '1rem', backgroundColor: '#f0f0f0', marginRight: '1rem'}}>
        <span style={{ marginRight: '1rem', fontWeight: 'bold'}}> GSB Frais </span>
        <Link to="/" style={{ marginRight: '1rem' }}>Accueil</Link>
        <Link to="/login" style={{ marginRight: '1rem' }}>Connexion</Link>
        <Link to="/dashboard">Tableau de bord</Link>
      </div>

      {/* Routes de l'application */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/dashboard" element={<Dashboard />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Étape 3 : Création de la barre de menu

Pour rappel, avec React, chacune des parties de l'interface (la barre de menu, les frais, le pied de page) qu'on va pouvoir réutiliser correspond à un composant. Ils contiennent tout ce qui est nécessaire à leur bon fonctionnement : **la structure, les styles et le comportement** (par exemple, les actions qui sont déclenchées quand on clique dessus).

Les composants nous permettent d'utiliser la même structure de données, et de remplir ces structures avec différents jeux de données. Peu importe le nombre de frais que vous aurez dans votre base, vous pourrez les exploiter pour afficher vos données sans aucun effort. Et si dans le futur, vous avez besoin de créer une nouvelle page avec la même présentation, vous pourrez réutiliser le même composant.

Remarque : les pages sont des composants aussi mais on ne les met pas dans le répertoire components car ce sont des composants de haut niveau qui représentent une route complète de l'application, une vue principale (ce que l'utilisateur voit quand il navigue) alors que les composants du répertoire components sont des briques de construction réutilisables dans plusieurs pages. On peut comparer au milieu de la construction :

- **Composants** = Briques, portes, fenêtres (réutilisables dans plusieurs maisons).
- **Pages** = Maisons complètes (chaque maison = une route, composée de briques/composants).

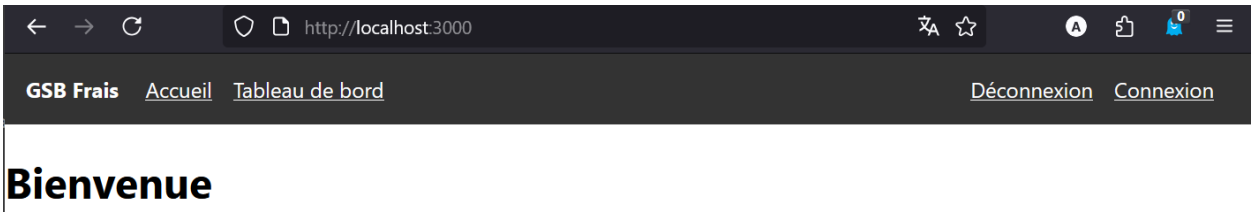
Nous allons commencer par créer le composant de la barre de navigation.

1. **Créer le composant Navbar** :
    - Dans /src/components/Navbar.jsx, créez un composant Navbar (sous la forme d'une fonction) avec :
        - Un lien vers la page d'accueil (/).
        - Un lien vers le tableau de bord (/dashboard) **uniquement si l'utilisateur est connecté**.
        - Un bouton de connexion ou de déconnexion selon l'état de l'utilisateur.

```
function Navbar() {
  return (
    <nav style={{ padding: '1rem', backgroundColor: '#333', color: 'white' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between' }}>
        <div>
          <Link to="/" style={{ color: 'white', marginRight: '1rem' }}>Accueil</Link>
          <Link to="/dashboard" style={{ color: 'white' }}>Tableau de bord</Link>
        </div>
        <div>
          <Link to="" style={{ color: 'white' }}>Déconnexion</Link>
            <Link to="/login" style={{ color: 'white' }}>Connexion</Link>
        </div>
      </div>
    </nav>
  );
}
export default Navbar;
```

    - Ne pas oublier d'importer Link depuis react-router-dom

2. **Supprimer le CSS inline** (écrit directement dans les balises JSX) et importer le style depuis un fichier Navbar.css (à créer dans le dossier /src/styles) contenant les classes nécessaires.
3. **Intégrer Navbar dans App.js** :
   - Dans App.js, supprimer les liens temporaires et placer le composant Navbar afin qu'il s'affiche en haut de toutes les pages.
4. Vérifier l'affichage :



Cette barre de navigation n'est pas optimale, il manque deux fonctionnalités :

- Le lien vers le tableau de bord (/dashboard) devrait s'afficher **uniquement si l'utilisateur est connecté**.
- Un seul des deux boutons de connexion ou de déconnexion devrait s'afficher selon l'état de l'utilisateur.

Nous les ajouterons après avoir implémenté l'authentification.

Étape 4 : Gestion de l'authentification

Nous allons créer un contexte React (AuthContext) pour gérer l'état de l'authentification (utilisateur connecté ou non) et le partager entre tous les composants de l'application. Cela évite de passer manuellement les props (user, login, logout) à chaque composant.

1. **Créer le contexte d'authentification en complétant le code suivant** (au niveau de « ToDo ») dans /src/context/AuthContext.js.
   - Ce code crée un contexte AuthContext avec :
     - Un état `user` (initialement null) pour stocker l'utilisateur connecté.
     - Deux fonctions : `loginUser` et `logoutUser`.
   - AuthProvider est exporté et fournit les valeurs user, login et logout à tous les composants enfants.
   - On crée également un hook personnalisé useAuth qui permet d'accéder facilement au contexte depuis n'importe quel composant. On pourra l'utiliser comme suit dans n'importe quel composant : `const { user, loginUser, logoutUser } = useAuth();`

```
2. import { createContext, useContext, useState } from 'react';
3.
4. // 1. Création du contexte
```

```
5. const AuthContext = createContext();
6.
7. // 2. Fournisseur du contexte (AuthProvider)
8. export function AuthProvider({ children }) {
9.     // État local pour stocker l'utilisateur (null = non connecté)
10.    // ToDo : affecter la valeur nulle
11.
12.  // 3. Fonction de connexion
13.  const loginUser = (login, password) => {
14.    // ToDo : implémenter une connexion avec une logique simplifiée :
   vérifie
15.    // si login/mot de passe correspondent à des valeurs prédéfinies
16.    // Si le login vaut "Andre" et le mot de passe vaut "secret"
17.    // alors renvoyer true et mettre à jour l'état avec le login de
   l'utilisateur connecté
18.    // Si la connexion échoue, renvoyer false
19.  };
20.
21.  // 4. Fonction de déconnexion
22.  const logoutUser = () => {
23.    // ToDo : réinitialiser la valeur de l'état à null
24.  };
25.
26.  // 5. Valeurs exposées aux composants enfants
27.  return (
28.    <AuthContext.Provider value={{ user, loginUser, logoutUser }}>
29.      {children} {/* Rend les composants enfants (ex: App) */}
30.    </AuthContext.Provider>
31.  );
32.}
33.
34.// 6. Hook personnalisé pour utiliser le contexte facilement
35.export function useAuth() {
36.  return useContext(AuthContext);
37.}
38.
```

Le fonctionnement sera le suivant :

```
AuthProvider (dans App.js)
│
├── Met à disposition : { user, login, logout }
│
├── Composant A (ex: Navbar)
│   └── Utilise `useAuth()` pour afficher/masquer des liens
│
├── Composant B (ex: Login)
│   └── Utilise `useAuth()` pour appeler `login(email, password)`
│
└── Composant C (ex: Dashboard)
    └── Utilise `useAuth()` pour vérifier si `user` est connecté
```

2. **Envelopper l'application avec AuthProvider** :

   - Dans App.js, enveloppez les routes avec <AuthProvider>.

```
<AuthProvider>
  <Router>
    {/* ... */}
  </Router>
</AuthProvider>
```

3. **Créer la page de connexion (Login.jsx)** :

   - Importer les fonctions nécessaires : useState pour stocker le login et le mot de passe dans des états, useAuth pour accéder à notre fonction login, useNavigate pour rediriger vers le dashboard après connexion.

```
import { useState } from 'react';
import { useAuth } from '../context/AuthContext';
import { useNavigate } from 'react-router-dom';
```

   - Compléter le code suivant (au niveau de « ToDo ») pour que le formulaire contienne deux champs : login et password, et un bouton de validation.

```
export default function Login() {
  // 1. États locaux pour les champs du formulaire
  const [login, setLogin] = useState('');
  const [password, setPassword] = useState('');


  // 5. Rend le formulaire
  return (
    <div className="login-page">
      <div className="login-container">
        <h1>Connexion</h1>
      <form className="login-form" onSubmit={handleSubmit}>
        <div>
          <label>Login :</label>
          <input
            type="text"
            value={login}
            onChange={(e) => setLogin(e.target.value)}
            required
          />
        </div>
        // TODO : compléter le formulaire de connexion
      </form>
    </div>
    </div>
  );
}
```

- Utilisez la fonction loginUser du contexte pour simuler une connexion :
  - Après la définition des états login et password, déclarer la variable loginUser (type const), et lui affecter la valeur de retour de l'appel à la fonction useAuth.
  - Avec le code suivant, définir la variable navigate et la fonction handleSubmit qui vérifie que la fonction login renvoie true pour rediriger vers le dashboard, et affiche le message 'Identifiants incorrects' sinon.

```
// 3. Hook pour la redirection après connexion
const navigate = useNavigate();

// 4. Déclaration de la fonction handleSubmit
const handleSubmit = (e) => {
  e.preventDefault(); // Empêche le rechargement de la page
  // Appel de la fonction login avec email et password
  if (loginUser(login, password)) {
    navigate('/dashboard'); // Redirige vers /dashboard si succès
  } else {
    alert('Identifiants incorrects'); // Affiche une erreur si échec
  }
};
```

- Appeler la fonction handleSubmit à la soumission du formulaire :

```
<form onSubmit={handleSubmit}>
```

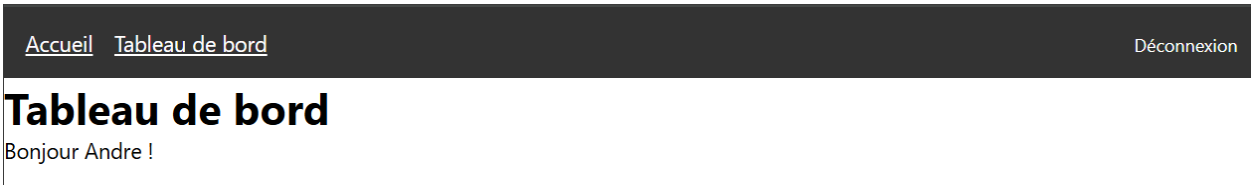
  - Redirigez vers /dashboard après une connexion réussie.
  - Vérifiez que lorsque vous rentrez le bon login et mot de passe dans le formulaire, vous êtes bien redirigé vers le dashboard.

4. **Modifier la barre de navigation avec la gestion de la connexion:**
   - Dans Navbar.jsx, récupérer user et logoutUser via un appel à notre hook useAuth.
   - Appeler la fonction logoutUser du contexte lors du clic sur "Déconnexion" en remplaçant la balise Link par une balise button (code à compléter ci-dessous) :

```
<button onClick={} style={{ color: 'white', background: 'none', border: 'none' }}>Déconnexion</button>
```

   - Gérer le rendu conditionnel du menu de navigation (cf document 5_Props_application du cours sur l'appli Sneakers). Si user est défini, afficher le lien Tableau de bord et le bouton Déconnexion. Sinon, afficher le lien Connexion.
5. Afficher un message personnalisé à l'utilisateur dans le tableau de bord : « Bienvenue » suivi du login de l'utilisateur connecté.
6. Vérifier le bon fonctionnement en vous connectant, puis en allant sur le dashboard et en vous déconnectant depuis le dashboard. Vous devez obtenir les affichages suivants :

Après connexion :



Après déconnexion :

7. Améliorer le style :
   - Copiez le fichier Login.css fourni dans votre dossier src/styles
   - Importez Login.css dans Login.jsx

- Vérifiez que vous obtenez le rendu suivant :

Étape 5 : Créer une route protégée pour empêcher l'accès à /dashboard sans authentification.

Nous allons créer un composant PrivateRoute pour encapsuler le composant Dashboard et conditionner son affichage. Dashboard sera donc un composant enfant (children) de PrivateRoute.

Pour rappel, en React, **children** est une prop spéciale qui représente tout ce qui est placé entre les balises d'ouverture et de fermeture d'un composant.

1. Dans App.js, encapsulez le composant <Dashboard> dans <PrivateRoute>.
2. Créez un composant PrivateRoute qui reçoit children en prop et décide de l'afficher ou non :
   - Dans /src/components/PrivateRoute.jsx, utilisez le hook useAuth pour vérifier si l'utilisateur est connecté.
   - Si oui, renvoyez la page demandée (children), sinon redirigez vers /login en renvoyant <Navigate to='/login' />
3. Testez : normalement, si vous essayez d'afficher le tableau de bord sans être connecté (en tapant directement l'url http://localhost:3000/dashboard puisque vous n'avez pas le bouton), vous devez être redirigé vers le formulaire de connexion.